The University of Texas at Dallas

Airline Database System

Project Report

Group-7

Course Title: *Database Foundations for Business Analytics*

Course Code: *BUAN6320.002*

Prepared for:

Prof. Kannan Srikanth

Prepared by:

Bhoomi Parikh – BBP240001

Sai Nischala Kuchibhotla – SXK230312

Vamsi Krishna Kovi – VXK240023

Nandhakumar Sivakumar – NXS230144

Harsha Vardhana Reddy Diddala- HVD240000

Date: 4th December 2024

BUAN 6320 - Database Foundations for Business Analytics Fall 2024

Table of contents

S.no	Content	Page Number
1.	Introduction	3
2.	Meeting Minutes	4
3.	Project Charter	5
4.	Project Writeup	6
5.	Database Tables	9
6.	Complex Queries	10
7.	Stored Procedures	17
8.	Stored Functions	23
9.	Triggers	26
10.	Conclusion	32

Introduction

The Airline Database Management System is designed to streamline and enhance the operations of an airline by effectively managing essential data related to flights, passengers, reservations, crew, and aircraft. This system serves as the backbone for ensuring efficient data storage, retrieval, and processing, enabling the airline to provide seamless services to its customers and maintain operational excellence.

In a highly competitive and dynamic industry, the efficient management of information is critical for decision-making, operational efficiency, and customer satisfaction. The proposed database system integrates various aspects of airline operations into a cohesive framework, offering functionalities like reservation management, flight scheduling, crew assignment, and real-time data accessibility.

By leveraging a robust database design, this system ensures data consistency, accuracy, and security, addressing the unique challenges of the aviation sector. It lays the foundation for advanced analytics and automation, paving the way for future innovation and scalability.

Meeting Minutes

Meeting Data	Members	Deliverables
09/26/24	Vamsi Krishna Kovi, Bhoomi Parikh, Sai Nischala Kuchibhotla, Harsha Diddala , Nandhakumar Sivakumar (10mins)	Detailed requirements about the project
09/29/24	Bhoomi Parikh, Sai Nischala	Project Charter

	Kuchibhotla, Vamsi Krishna Kovi (2.5 hrs)	
10/09/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi (2.5 hrs)**	**Create 20-25 Tables**
10/17/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi,** Nandhakumar Sivakumar **(2.5 hrs)**	**Database Design**
10/23/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi (2.5 hrs)**	**ER diagram, data dictionary**
10/30/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi (2.5 hrs)**	**10 complex queries**
11/07/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla(2.5 hrs)**	**5 Stored Procedures**
11/13/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi (2.5 hrs)**	**5 Functions**
11/25/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla(2.5 hrs)**	**5 Triggers**
12/05/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi.** Nandhakumar Sivakumar, Harsha Vardhana Reddy Diddala **(5 hrs)**	**Recording the Video Presentation**
12/07/24	**Bhoomi Parikh, Sai Nischala Kuchibhotla, Vamsi Krishna Kovi(7hrs)**	**Final Editing Video and Drafting Project Report**

Project Charter

- **Project Title :** Airline System Database
- **Prepared by :**

  Bhoomi Parikh
  Sai Nischala Kuchibhotla

  Vamsi Krishna Kovi

  Nandhakumar Sivakumar

  Harsha Vardhana Reddy Diddala

- **Date :** 09/29/2024
- **Project Objective :**
  - Enhance data accessibility and reliability
  - Streamline processe
  - Improve decision-making
  - Centralize information
  - Enhance customer service
- **Assumptions :**
  - Sufficient technical resources and expertise
  - Adequate budget allocation (hardware, software, personnel).
  - Compatibility with existing airline systems for integration
  - Skilled and experienced project team
  - On-time and on-budget project completion
- **Project Scope :**
  - Airline database system design and development
  - Data migration from legacy systems
  - Integration with booking and flight operation systems
  - User-friendly interfaces for access and reporting
  - Data security implementation

Project writeup

The Airline Database Management System (ADMS) is a centralized solution designed to streamline the management of airline operations and data, addressing the complexities and inefficiencies of traditional systems. The ADMS automates key processes, ensures data

accuracy, and enhances decision-making capabilities for airline management. It securely organizes critical information, including flight schedules, passenger records, crew assignments, and maintenance logs, providing seamless access for authorized users.

Key features of the ADMS include role-based access control to safeguard sensitive data, automated processes for reservation management, flight scheduling, and reporting, as well as robust analytics tools that offer insights into operational trends and passenger behavior. These features minimize errors, reduce manual data entry, and save time for airline staff.

The system is designed to be scalable and adaptable, capable of accommodating growing passenger numbers and evolving airline operations. Advanced data security measures ensure compliance with aviation regulations, protecting passenger and operational data. Additionally, by offering real-time access and actionable insights, the ADMS empowers airline administrators to make informed decisions and improve overall efficiency.

The ADMS not only enhances operational workflows but also improves the customer experience by ensuring accurate and timely information. It aligns with the needs of modern aviation, providing a reliable, secure, and efficient solution for managing airline data. The project includes several key milestones to ensure timely and efficient delivery of its components:

Project Initialization and Requirement Gathering (09/25/2024): Completing the scoping phase, gathering requirements, and defining objectives for the ADMS.

Completion of Requirement Analysis (09/28/2024): Finalizing the scope and preparing a detailed feature set document for the system.

ER Diagram Design (10/01/2024): Designing the entity-relationship diagram to represent the database structure for airline records.

System Development Phase 1 (10/05/2024): Developing basic modules for flight data entry, updates, deletion, and comprehensive record listing.

System Development Phase 2 (10/20/2024): Implementing role-based access control, automated reporting, and analytics functionalities.

Testing and Quality Assurance (11/02/2024): Conducting rigorous testing to ensure system reliability, data accuracy, and user satisfaction.

System Deployment and User Training (11/16/2024): Deploying the final system, providing training sessions, and delivering user manuals for staff and administrators

Impact Statement:

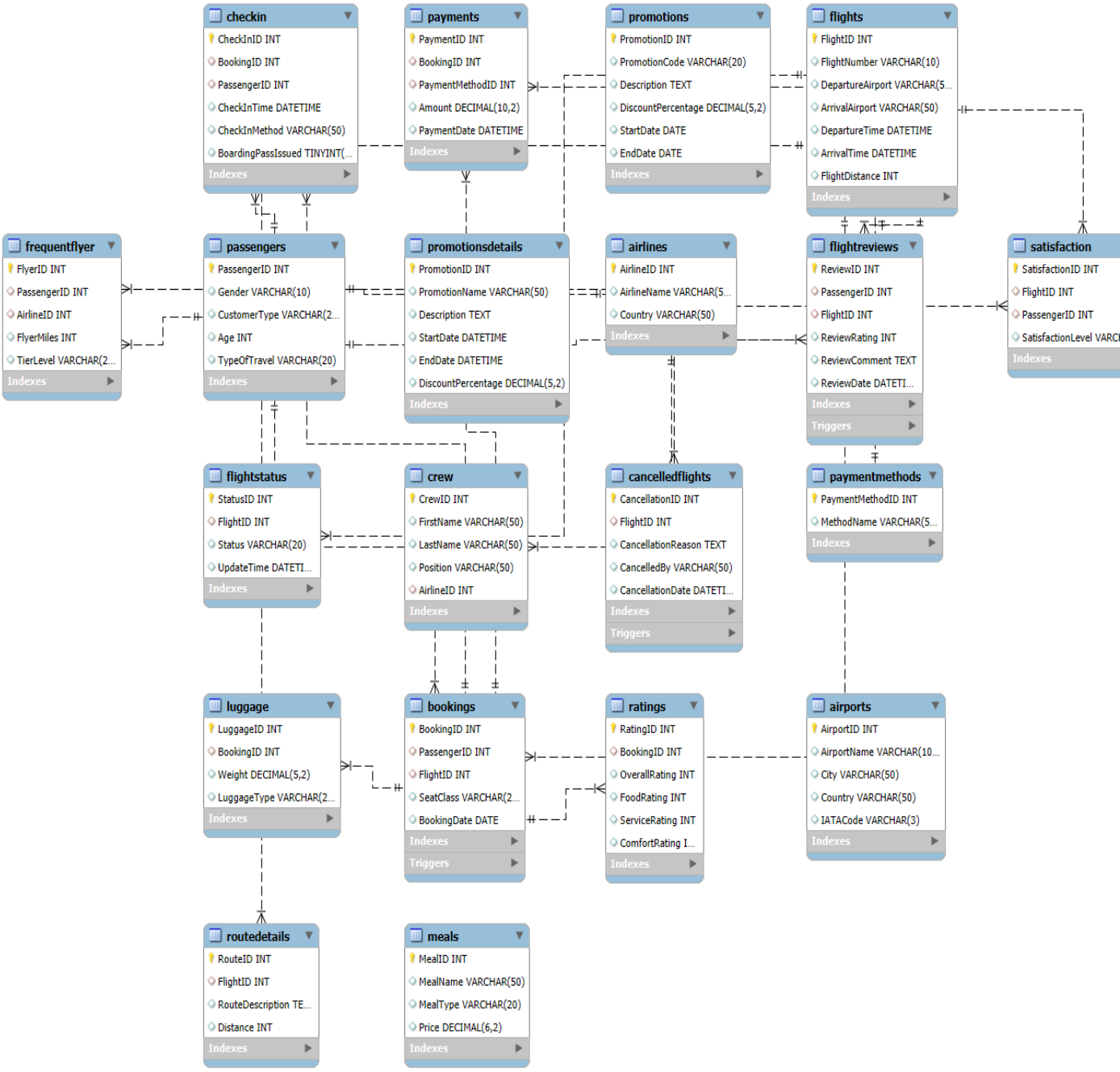
The ADMS addresses the increasing need for efficient and secure data management in the aviation industry. By automating routine tasks and providing advanced analytics, the system reduces errors, minimizes manual workload, and enhances operational decision-making. It

supports modern aviation trends that prioritize data-driven operations and technology integration.

Key impacts include enhanced workflows through automation, improved data accuracy, and compliance with data protection regulations. The system facilitates collaboration across departments, such as flight operations, customer service, and maintenance teams, ensuring seamless integration and user adoption.

The ADMS streamlines processes like reservation management, flight scheduling, and reporting while ensuring data security and regulatory compliance. Though initial investments in infrastructure and training are required, the long-term benefits in operational efficiency and improved customer service outweigh the costs. The ADMS provides a scalable, customizable, and forward-thinking solution to the critical challenges of managing airline operations effectively.

ER Diagram



Database Tables

Our database is made of the following tables:

1. **Passengers**: Stores details of passengers, including demographics and travel preferences.
2. **Flights**: Contains flight details, including schedules, airports, and distances.
3. **Bookings**: Records passenger bookings, linking them to flights and seat classes.
4. **Airlines**: Stores information about airline companies, including names and countries.
5. **Airports**: Contains details of airports, including location and IATA codes.
6. **FlightStatus**: Tracks the current status of flights (e.g., On Time, Delayed, Cancelled).
7. **Crew**: Stores crew member details, including positions and associated airlines.
8. **Luggage**: Tracks passenger luggage details, including weight and type.
9. **Meals**: Contains details of in-flight meal options and pricing.
10. **Ratings**: Records passenger ratings for flights, including food, service, and comfort.
11. **Promotions**: Stores promotional offers, including discounts and validity periods.
12. **PaymentMethods**: Lists available methods of payment for bookings.
13. **Payments**: Tracks payment details for bookings, including amounts and payment dates.
14. **Satisfaction**: Records passenger satisfaction levels for specific flights.
15. **FrequentFlyer**: Tracks frequent flyer details, including miles and tier levels.
16. **CancelledFlights**: Stores details of flight cancellations and reasons.
17. **RouteDetails**: Describes flight routes, including distance and route descriptions.
18. **FlightReviews**: Records passenger reviews and ratings for flights.
19. **CheckIn**: Tracks passenger check-in details, including methods and times.
20. **PromotionsDetails**: An alternative structure for storing promotional offers.

Complex Queries

1. Find the top 5 routes with the highest customer satisfaction:

This query identifies the top 5 flight routes with the highest average customer satisfaction rates, based on the proportion of satisfied passengers, and ranks them by satisfaction rates.

```
SELECT f.DepartureAirport, f.ArrivalAirport,
	AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
SatisfactionRate,
	COUNT(*) AS TotalPassengers
FROM Flights f
JOIN Satisfaction s ON f.FlightID = s.FlightID
GROUP BY f.DepartureAirport, f.ArrivalAirport
ORDER BY SatisfactionRate DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

DepartureAirport	ArrivalAirport	SatisfactionRate	TotalPassengers
New York	London	1.0000	1
Los Angeles	Sydney	1.0000	1
Chicago	Berlin	1.0000	1
Paris	Tokyo	0.0000	1
Dubai	Singapore	0.0000	1

2. Analyze the Impact of Flight Distance and Delay on Satisfaction Rates

This query analyzes how flight distance (short, medium, or long) and flight delay status influence customer satisfaction rates, providing insights into their combined impact.

```
SELECT
	CASE
	WHEN f.FlightDistance < 1000 THEN 'Short'
	WHEN f.FlightDistance BETWEEN 1000 AND 5000 THEN 'Medium'
	ELSE 'Long'
	END AS DistanceCategory,
	fs.Status AS DelayStatus,
	COUNT(s.SatisfactionLevel) AS TotalPassengers,
	AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) *
100 AS SatisfactionRate
FROM
	Flights f
JOIN
	FlightStatus fs ON f.FlightID = fs.FlightID
```

```
JOIN
        Satisfaction s ON f.FlightID = s.FlightID
GROUP BY
        DistanceCategory, DelayStatus
ORDER BY
        DistanceCategory, DelayStatus;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DistanceCategory	DelayStatus	TotalPassengers	SatisfactionRate
Long	Cancelled	1	0.0000
Long	Delayed	1	0.0000
Long	On Time	3	100.0000

3. Compare satisfaction rates between loyal and disloyal customers across different travel types

This query compares satisfaction rates of loyal and disloyal customers across different types of travel (business or personal), offering insights into customer behavior and preferences.

```
SELECT
        p.CustomerType,
        p.TypeOfTravel,
        AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
SatisfactionRate,
        COUNT(*) AS TotalPassengers
FROM Passengers p
JOIN Satisfaction s ON p.PassengerID = s.PassengerID
GROUP BY p.CustomerType, p.TypeOfTravel
ORDER BY p.CustomerType, p.TypeOfTravel;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerType	TypeOfTravel	SatisfactionRate	TotalPassengers
Disloyal Customer	Business travel	1.0000	3
Disloyal Customer	Personal Travel	0.0000	2

4. Find the Most Popular Routes by Frequent Flyer Tier

This query finds the most popular flight routes for each frequent flyer tier, ranked by the number of bookings, highlighting patterns in loyalty program usage.

```
SELECT
        ff.TierLevel,
        CONCAT(f.DepartureAirport, ' to ', f.ArrivalAirport) AS Route,
        COUNT(b.BookingID) AS TotalBookings
FROM
        FrequentFlyer ff
JOIN
        Passengers p ON ff.PassengerID = p.PassengerID
JOIN
        Bookings b ON p.PassengerID = b.PassengerID
JOIN
        Flights f ON b.FlightID = f.FlightID
GROUP BY
        ff.TierLevel, Route
ORDER BY
        ff.TierLevel DESC, TotalBookings DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

TierLevel	Route	TotalBookings
Silver	New York to London	1
Silver	Chicago to Berlin	1
Silver	Seoul to Moscow	1
Platinum	Dubai to Singapore	1
Platinum	Bangkok to San Francisco	1
Gold	Paris to Tokyo	1
Gold	Mumbai to Hong Kong	1
Gold	Amsterdam to Cape Town	1
Bronze	Los Angeles to Sydney	1
Bronze	Toronto to São Paulo	1

5. Analyze the relationship between age groups and satisfaction across different seat classes:

This query examines how satisfaction rates vary among different age groups and seat classes, identifying trends in passenger preferences.

```
SELECT
        CASE
        WHEN p.Age < 20 THEN 'Under 20'
        WHEN p.Age BETWEEN 20 AND 40 THEN '20-40'
        WHEN p.Age BETWEEN 41 AND 60 THEN '41-60'
        ELSE 'Over 60'
        END AS AgeGroup,
```

```
        b.SeatClass,
        AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
SatisfactionRate,
        COUNT(*) AS TotalPassengers
FROM Passengers p
JOIN Satisfaction s ON p.PassengerID = s.PassengerID
JOIN Bookings b ON s.FlightID = b.FlightID
GROUP BY AgeGroup, b.SeatClass
ORDER BY AgeGroup, b.SeatClass;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

AgeGroup	SeatClass	SatisfactionRate	TotalPassengers
20-40	Business	1.0000	2
20-40	Economy	0.0000	2
41-60	First Class	1.0000	1

6. Calculate the average ratings for each service category by customer type:

This query calculates the average ratings for food, service, and comfort categories for each customer type, providing an overview of service satisfaction by demographic.

```
SELECT
        p.CustomerType,
        AVG(r.FoodRating) AS AvgFoodRating,
        AVG(r.ServiceRating) AS AvgServiceRating,
        AVG(r.ComfortRating) AS AvgComfortRating
FROM Passengers p
JOIN Bookings b ON p.PassengerID = b.PassengerID
JOIN Ratings r ON b.BookingID = r.BookingID
GROUP BY p.CustomerType;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerType	AvgFoodRating	AvgServiceRating	AvgComfortRating
Disloyal Customer	3.1000	4.0000	3.6000

7. Identify flights with the highest dissatisfaction rates:

This query identifies flights with the highest rates of dissatisfied passengers, analyzing seat classes and flight distances to uncover potential issues.

```
SELECT
	f.FlightID,
	b.SeatClass,
	f.FlightDistance,
	COUNT(*) AS TotalPassengers,
	AVG(CASE WHEN s.SatisfactionLevel = 'Neutral or Dissatisfied' THEN 1.0
ELSE 0 END) AS DissatisfactionRate
FROM Flights f
JOIN Satisfaction s ON f.FlightID = s.FlightID
JOIN Bookings b ON f.FlightID = b.FlightID
GROUP BY f.FlightID, b.SeatClass, f.FlightDistance
ORDER BY DissatisfactionRate DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FlightID	SeatClass	FlightDistance	TotalPassengers	DissatisfactionRate
2	Economy	9713	1	1.00000
4	Economy	5841	1	1.00000
1	Business	5567	1	0.00000
3	First Class	12051	1	0.00000
5	Business	6856	1	0.00000

8. Analyze the impact of flight distance on satisfaction for different customer types:

This query explores the relationship between flight distance categories (short, medium, long) and satisfaction rates for different customer types.

```
SELECT
	p.CustomerType,
	CASE
	WHEN f.FlightDistance < 500 THEN 'Short'
	WHEN f.FlightDistance BETWEEN 500 AND 2000 THEN 'Medium'
	ELSE 'Long'
	END AS FlightDistanceCategory,
	AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
SatisfactionRate
FROM Passengers p
JOIN Satisfaction s ON p.PassengerID = s.PassengerID
JOIN Flights f ON s.FlightID = f.FlightID
GROUP BY p.CustomerType, FlightDistanceCategory
ORDER BY p.CustomerType, FlightDistanceCategory;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerType	FlightDistanceCategory	SatisfactionRate
Disloyal Customer	Long	0.6000

9. Total Luggage Weight by Seat Class and Flight

This query calculates the total luggage weight for each seat class on each flight, providing insights into baggage trends and operational planning.

```
SELECT
        f.FlightNumber,
        b.SeatClass,
        SUM(l.Weight) AS TotalLuggageWeight
FROM
        Flights f
JOIN
        Bookings b ON f.FlightID = b.FlightID
JOIN
        Luggage l ON b.BookingID = l.BookingID
GROUP BY
        f.FlightNumber, b.SeatClass
ORDER BY
        f.FlightNumber, TotalLuggageWeight DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FlightNumber	SeatClass	TotalLuggageWeight
FL001	Business	23.50
FL002	Economy	18.70
FL003	First Class	30.00
FL004	Economy	15.20
FL005	Business	22.80
FL006	Economy	10.50
FL007	Business	28.30
FL008	Economy	17.90
FL009	First Class	35.00
FL010	Economy	12.40

10. Identify the most common combinations of low-rated services:

This query identifies common combinations of low-rated services (food, service, and comfort) and their associated meal types, offering actionable insights for service improvement.

```
SELECT
	m.MealName AS FoodType,
	CASE WHEN r.FoodRating < 3 THEN 1 ELSE 0 END AS LowFoodRating,
	CASE WHEN r.ServiceRating < 3 THEN 1 ELSE 0 END AS
LowServiceRating,
	CASE WHEN r.ComfortRating < 3 THEN 1 ELSE 0 END AS
LowComfortRating,
	COUNT(*) AS Occurrences
FROM
	Ratings r
JOIN
	Bookings b ON r.BookingID = b.BookingID
JOIN
	Meals m ON b.BookingID = m.MealID  -- Assuming each booking has a meal
GROUP BY
	FoodType, LowFoodRating, LowServiceRating, LowComfortRating
ORDER BY
	Occurrences DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

FoodType	LowFoodRating	LowServiceRating	LowComfortRating	Occurrences
Chicken Pasta	0	0	0	1
Vegetarian Curry	1	0	0	1
Beef Steak	0	0	0	1
Vegan Salad	1	0	1	1
Seafood Platter	0	0	0	1
Gluten-free Sandwich	0	0	0	1
Kosher Meal	0	0	0	1
Halal Lamb	1	0	0	1
Fruit Platter	0	0	0	1
Kids Meal	1	0	1	1

Stored Procdures

1. Calculate overall satisfaction score for a specific flight:

This procedure calculates the overall satisfaction score for a specific flight by averaging food, service, comfort, and overall ratings. It provides a single aggregated value reflecting passenger satisfaction for the flight.

```
DELIMITER //

CREATE PROCEDURE CalculateFlightSatisfaction(IN flightID INT, OUT
satisfactionScore DECIMAL(4,2))

BEGIN

        SELECT AVG(

        r.FoodRating + r.ServiceRating + r.ComfortRating +

        IFNULL(r.OverallRating, 0)

        ) / 4 INTO satisfactionScore

        FROM Ratings r

        JOIN Bookings b ON r.BookingID = b.BookingID

        WHERE b.FlightID = flightID;

END //

DELIMITER ;


SET @satisfactionScore = 0;

CALL CalculateFlightSatisfaction(4, @satisfactionScore);

SELECT @satisfactionScore AS SatisfactionScore;
```

	SatisfactionScore
▶	2.00

2. Get satisfaction history of a specific passenger:

This procedure retrieves the satisfaction history of a specific passenger, including flight details, seat class, and satisfaction level. The data is ordered by the most recent flights.

```
DELIMITER //
CREATE PROCEDURE GetPassengerSatisfactionHistory(IN passengerID INT)
BEGIN
	SELECT f.FlightID, b.SeatClass, s.SatisfactionLevel, f.DepartureTime
	FROM Flights f
	JOIN Satisfaction s ON f.FlightID = s.FlightID
	JOIN Bookings b ON s.PassengerID = b.PassengerID
	WHERE s.PassengerID = passengerID
	ORDER BY f.DepartureTime DESC;
END //
DELIMITER ;

CALL GetPassengerSatisfactionHistory(3);
```

	FlightID	SeatClass	SatisfactionLevel	DepartureTime
▶	3	First Class	Satisfied	2024-11-03 23:15:00

3. Generate satisfaction report for a specific date range

Generates a report on satisfaction metrics for flights within a specific date range, including passenger counts, satisfaction rates, and average ratings for food, service, and comfort.

```
DELIMITER //
CREATE PROCEDURE GenerateSatisfactionReport(IN startDate DATE, IN endDate DATE)
BEGIN
	SELECT
	b.SeatClass,
	COUNT(*) AS TotalPassengers,
	AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS SatisfactionRate,
	AVG(r.FoodRating) AS AvgFoodRating,
```

```
        AVG(r.ServiceRating) AS AvgServiceRating,
        AVG(r.ComfortRating) AS AvgComfortRating
        FROM Flights f
        JOIN Satisfaction s ON f.FlightID = s.FlightID
        JOIN Bookings b ON f.FlightID = b.FlightID
        JOIN Ratings r ON b.BookingID = r.BookingID
        WHERE DATE(f.DepartureTime) BETWEEN startDate AND endDate
        GROUP BY b.SeatClass
        ORDER BY SatisfactionRate DESC;
END //
DELIMITER ;
```

SeatClass	TotalPassengers	SatisfactionRate	AvgFoodRating	AvgServiceRating	AvgComfortRating
Business	2	1.0000	3.5000	4.5000	4.0000
First Class	1	1.0000	5.0000	5.0000	5.0000
Economy	2	0.0000	1.5000	3.5000	2.5000

4. Identify and report on frequent complainers:

Identifies passengers who frequently report dissatisfaction, providing their complaint count and average ratings for flights.

```
DELIMITER //
CREATE PROCEDURE IdentifyFrequentComplainers(IN complaintThreshold INT)
BEGIN
        SELECT
        p.PassengerID,
        p.CustomerType,
        COUNT(*) AS TotalFlights,
        SUM(CASE WHEN s.SatisfactionLevel = 'Neutral or Dissatisfied' THEN 1
ELSE 0 END) AS Complaints,
        AVG(
        r.FoodRating + r.ServiceRating + r.ComfortRating +
        IFNULL(r.OverallRating, 0)
        ) / 4 AS AvgRating
```

```
    FROM Passengers p
    JOIN Satisfaction s ON p.PassengerID = s.PassengerID
    JOIN Bookings b ON s.FlightID = b.FlightID AND s.PassengerID =
b.PassengerID
    JOIN Ratings r ON b.BookingID = r.BookingID
    GROUP BY p.PassengerID, p.CustomerType
    HAVING Complaints >= complaintThreshold
    ORDER BY Complaints DESC;
END //
DELIMITER ;

CALL IdentifyFrequentComplainers(1);
```

	PassengerID	CustomerType	TotalFlights	Complaints	AvgRating
▶	2	Disloyal Customer	1	1	3.00000000
	4	Loyal Customer	1	1	2.00000000

5. Update customer type based on flight frequency:

Automatically updates the customer type based on the number of flights taken. Passengers with more than 10 flights are labeled as "Loyal Customer"; others as "Disloyal Customer."

```
DELIMITER //
CREATE PROCEDURE UpdateCustomerType()
BEGIN
    UPDATE Passengers p
    JOIN (
    SELECT PassengerID, COUNT(*) AS FlightCount
    FROM Bookings
    GROUP BY PassengerID
    ) b ON p.PassengerID = b.PassengerID
    SET p.CustomerType =
    CASE
```

```
        WHEN b.FlightCount > 10 THEN 'Loyal Customer'
        ELSE 'Disloyal Customer'
        END;
END //
DELIMITER ;

SELECT PassengerID, CustomerType FROM Passengers;
```

PassengerID	CustomerType
1	Loyal Customer
2	Disloyal Customer
3	Loyal Customer
4	Loyal Customer
5	Disloyal Customer

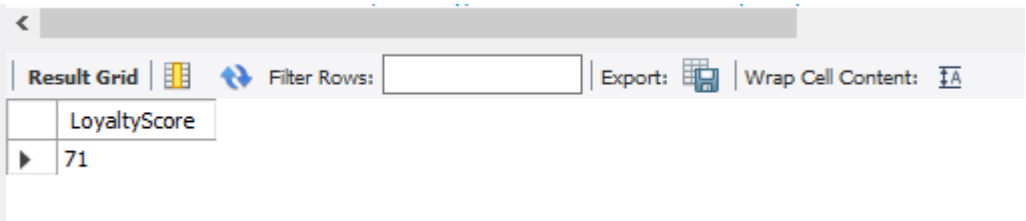
Stored Functions

1. Calculate customer loyalty score

```
DELIMITER //
CREATE FUNCTION CalculateCustomerLoyaltyScore(passengerID INT)
RETURNS INT
DETERMINISTIC
BEGIN
   DECLARE loyaltyScore INT;

   SELECT
      COUNT(*) * 10 +
      SUM(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 5 ELSE 0 END) +
      SUM(f.FlightDistance / 100) INTO loyaltyScore
   FROM Satisfaction s
   JOIN Flights f ON s.FlightID = f.FlightID
   WHERE s.PassengerID = passengerID;

   RETURN COALESCE(loyaltyScore, 0);
END //
DELIMITER ;
```

Query to call function:

```
SELECT CalculateCustomerLoyaltyScore(1) AS LoyaltyScore;
```



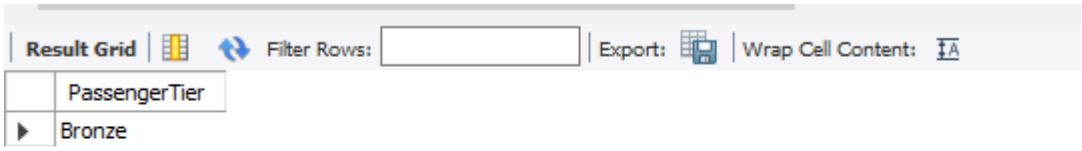
2. Determine passenger tier based on loyalty score

```
DELIMITER //
CREATE FUNCTION DeterminePassengerTier(loyaltyScore INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
```

```
    DECLARE tier VARCHAR(20);

    IF loyaltyScore > 1000 THEN
        SET tier = 'Platinum';
    ELSEIF loyaltyScore > 500 THEN
        SET tier = 'Gold';
    ELSEIF loyaltyScore > 200 THEN
        SET tier = 'Silver';
    ELSE
        SET tier = 'Bronze';
    END IF;

    RETURN tier;
END //
DELIMITER ;
```

Query to call function:

```
SELECT DeterminePassengerTier(72) AS PassengerTier;
```



3. Calculate flight punctuality percentage

```
DELIMITER //
CREATE FUNCTION CalculateFlightPunctuality(flightID INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE totalFlights INT DEFAULT 0;
    DECLARE onTimeFlights INT DEFAULT 0;
    DECLARE punctualityPercentage DECIMAL(5,2);

    -- Assuming there is a column DepartureDelayMinutes in the FlightStatus table to
check delay times
    SELECT COUNT(*) INTO totalFlights
    FROM FlightStatus
    WHERE FlightID = flightID;
```

```
    SELECT COUNT(*) INTO onTimeFlights
    FROM FlightStatus
    WHERE FlightID = flightID AND Status = 'On Time';

    SET punctualityPercentage = (onTimeFlights * 100.0) / GREATEST(totalFlights,
1);

    RETURN punctualityPercentage;
END //
DELIMITER ;
```

Query to call function:

```
SELECT CalculateFlightPunctuality(2) AS PunctualityPercentage;
```

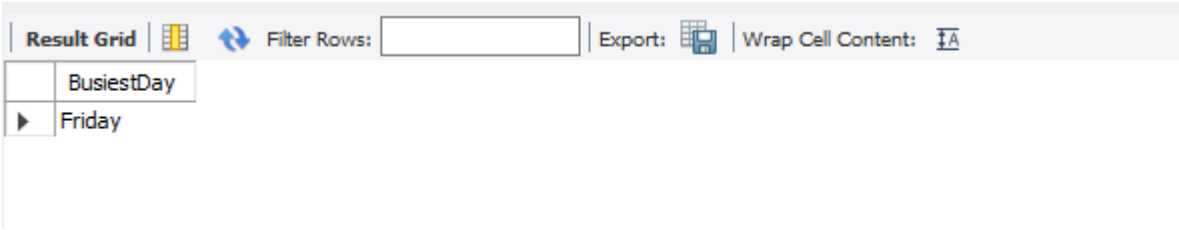
4. Determine the busiest day of the week for flights:

```
DELIMITER //
CREATE FUNCTION GetBusiestDayOfWeek()
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    DECLARE busiestDay VARCHAR(10);

    SELECT
        CASE DayOfWeek
            WHEN 1 THEN 'Sunday'
            WHEN 2 THEN 'Monday'
            WHEN 3 THEN 'Tuesday'
            WHEN 4 THEN 'Wednesday'
            WHEN 5 THEN 'Thursday'
            WHEN 6 THEN 'Friday'
            WHEN 7 THEN 'Saturday'
        END AS DayOfWeekName
    INTO busiestDay
    FROM (
        SELECT DAYOFWEEK(DepartureTime) AS DayOfWeek, COUNT(*) AS
FlightCount
        FROM Flights
        GROUP BY DAYOFWEEK(DepartureTime)
```

```
        ORDER BY FlightCount DESC
        LIMIT 1
    ) AS SubQuery;

    RETURN busiestDay;
END //
DELIMITER ;
```

Query to call function:

```
SELECT GetBusiestDayOfWeek() AS BusiestDay;
```



Triggers

1. Update CustomerType Based on Total Flights Booked

```
DELIMITER //

CREATE TRIGGER UpdateCustomerTypeAfterBooking

AFTER INSERT ON Bookings

FOR EACH ROW

BEGIN

    DECLARE flight_count INT;


    -- Count the total bookings for the passenger

    SELECT COUNT(*) INTO flight_count

    FROM Bookings

    WHERE PassengerID = NEW.PassengerID;
```

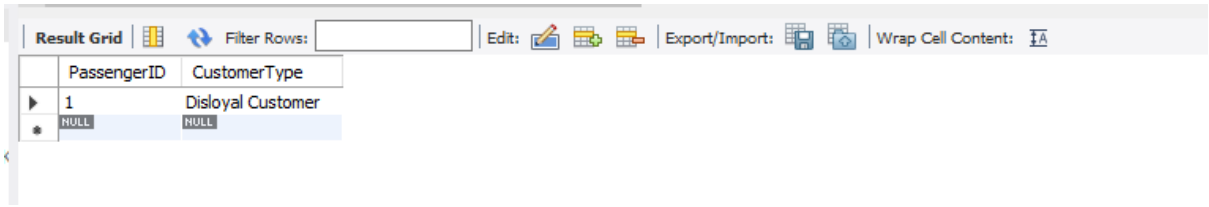
```
    -- Update CustomerType based on the number of bookings
    IF flight_count > 10 THEN
        UPDATE Passengers
        SET CustomerType = 'Loyal Customer'
        WHERE PassengerID = NEW.PassengerID;
    ELSE
        UPDATE Passengers
        SET CustomerType = 'Disloyal Customer'
        WHERE PassengerID = NEW.PassengerID;
    END IF;
END;
//
DELIMITER ;
```

Insert a booking to test the trigger

```
INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate)
VALUES (1, 2, 'Economy', '2024-12-01');
```

Verify that the CustomerType has been updated

```
SELECT PassengerID, CustomerType FROM Passengers WHERE PassengerID = 1;
```



2. Track CancellationReason in FlightReviews Table for Cancelled Flights

```
DELIMITER //
CREATE TRIGGER AddCancellationToReviews
AFTER INSERT ON CancelledFlights
FOR EACH ROW
BEGIN
```

```
    INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment,
ReviewDate)

    VALUES (NULL, NEW.FlightID, 1, CONCAT('Flight cancelled due to: ',
NEW.CancellationReason), NEW.CancellationDate);

END;

//

DELIMITER ;
```

Insert a cancelled flight to test the trigger

```
INSERT INTO CancelledFlights (FlightID, CancellationReason, CancelledBy,
CancellationDate)

VALUES (3, 'Technical issues', 'Maintenance', '2024-12-01 10:00:00');
```

Verify that the cancellation reason has been added to FlightReviews

```
SELECT * FROM FlightReviews WHERE FlightID = 3;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ReviewID	PassengerID	FlightID	ReviewRating	ReviewComment	ReviewDate
3	3	3	3	Average flight, could be better.	2024-11-03 06:00:00
11	NULL	3	1	Flight cancelled due to: Technical issues	2024-12-01 10:00:00
NULL	NULL	NULL	NULL	NULL	NULL

3. Prevent Booking on Cancelled Flights

```
DELIMITER //

CREATE TRIGGER PreventBookingOnCancelledFlights

BEFORE INSERT ON Bookings

FOR EACH ROW

BEGIN

    DECLARE is_cancelled INT;


    -- Check if the flight is marked as cancelled in the CancelledFlights table

    SELECT COUNT(*) INTO is_cancelled

    FROM CancelledFlights
```

```
    WHERE FlightID = NEW.FlightID;

    -- If the flight is cancelled, prevent booking and raise an error
    IF is_cancelled > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking on a cancelled flight is not allowed.';
    END IF;
END;
//
DELIMITER ;
```

Attempt to insert a booking for a cancelled flight

```
INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate)
VALUES (2, 3, 'Business', '2024-12-02');
```

Expected result: Error message - "Booking on a cancelled flight is not allowed."

⊗	30	15:56:53	INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate) VALUES (2, 3, 'Busi...	Error Code: 1644. Booking on a cancelled flight is not allowed.

---

4. Prevent Duplicate Passenger Reviews for the Same Flight

```
DELIMITER //
CREATE TRIGGER PreventDuplicateFlightReview
BEFORE INSERT ON FlightReviews
FOR EACH ROW
BEGIN
    DECLARE review_count INT;

    -- Check if the passenger has already submitted a review for the flight
    SELECT COUNT(*) INTO review_count
    FROM FlightReviews
    WHERE PassengerID = NEW.PassengerID AND FlightID = NEW.FlightID;
```

```
    -- If a review already exists for this passenger and flight, prevent insertion and raise an error

    IF review_count > 0 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Duplicate review not allowed for the same passenger and flight.';

    END IF;

END;

//

DELIMITER ;
```

Insert a review for a specific flight and passenger

```
INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment, ReviewDate)

VALUES (1, 2, 5, 'Excellent flight!', '2024-12-01 14:00:00');
```

Attempt to insert a duplicate review for the same flight and passenger

```
INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment, ReviewDate)

VALUES (1, 2, 4, 'Good flight overall.', '2024-12-02');
```

Expected result: Error message - "Duplicate review not allowed for the same passenger and flight."

32	15:58:17	INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment, Review...	Error Code: 1644. Duplicate review not allowed for the same passenger and flight.

5. Automatically Update Customer Type to "Frequent Traveler" After 10 Bookings

```
DELIMITER //

CREATE TRIGGER UpdateFrequentTravelerStatus

AFTER INSERT ON Bookings

FOR EACH ROW

BEGIN
```

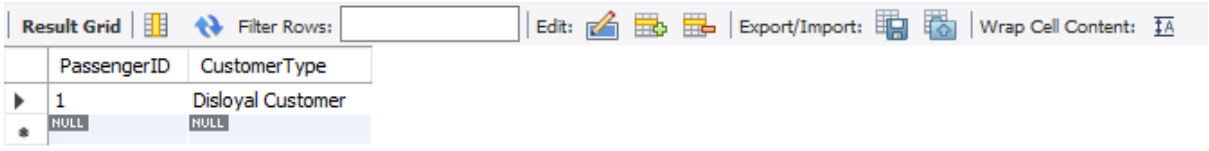
```
    DECLARE booking_count INT;

    -- Count the total bookings for the passenger
    SELECT COUNT(*)
    INTO booking_count
    FROM Bookings
    WHERE PassengerID = NEW.PassengerID;

    -- If the passenger has 10 or more bookings, update CustomerType to "Frequent Traveler"
    IF booking_count >= 10 THEN
        UPDATE Passengers
        SET CustomerType = 'Frequent Traveler'
        WHERE PassengerID = NEW.PassengerID;
    END IF;
END;
//
DELIMITER ;
```

Insert a booking to test the trigger

```
INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate)
VALUES (1, 6, 'Economy', '2024-12-02');
```

Verify that the CustomerType has been updated to "Frequent Traveler"

```
SELECT PassengerID, CustomerType FROM Passengers WHERE PassengerID = 3;
```



PassengerID	CustomerType
1	Disloyal Customer
NULL	NULL

Conclusion

The Airline Database Management System (ADMS) represents a robust and scalable solution tailored to meet the complex operational needs of modern airlines. By integrating essential components such as passenger management, flight scheduling, booking systems, crew assignments, and operational analytics, the ADMS streamlines workflows, enhances data accuracy, and improves decision-making capabilities.

The system’s modular design ensures flexibility, allowing airlines to adapt to evolving industry demands and technological advancements. Advanced security measures protect sensitive data, ensuring compliance with global regulations and safeguarding passenger trust.

By automating routine tasks, providing actionable insights, and enabling seamless communication across departments, the ADMS not only enhances operational efficiency but also elevates the customer experience. As a forward-thinking tool, it positions airlines to operate more effectively in an increasingly competitive and dynamic aviation landscape, driving long-term growth and sustainability.